UltraBERT-JEV: Encoder models for System One Applications

Astarag Mohapatra

Independent Researcher

astarag2843@gmail.com

November 12, 2026

Abstract

A decision model picks one answer from a set of options in a single forward pass, without writing out its reasoning. It is cheap enough to run on every item in a large pipeline, for example to grade, filter or route text. This technical report compares recipes for fine-tuning encoder-style decision models. We fine-tune UltraBERT-380M [32], an encoder pretrained with masked language modelling, with one mask per option. We compare four recipes, which differ in how each option is scored and how the scores are turned into probabilities over the options; one of them follows the expert gate of DeepSeek-V3 [14]. Against Qwen3-0.6B, fully fine-tuned on the same data with the LLM-as-Jev recipe [27] (**67.6%** test accuracy), the encoder reaches **69.7%** to **72.1%**. Among the encoder recipes, scoring an option by "yes" minus "no" instead of "yes" alone raises test accuracy from 69.7% to 71.3% and external-benchmark accuracy from 57.8% to **59.6%** (**56.1%** for the decoder). The sigmoid normalisation adds 0.8 test points and nothing on the external benchmarks; the square-root softplus adds nothing. One request at a time on an NVIDIA A100 80GB GPU, the encoder takes **29 ms** per decision against **68 ms** for Qwen3-0.6B, with similar peak memory. We also add an auxiliary head that predicts, in the same pass, the sentences annotated as supporting the answer. It is trained on human-marked evidence and on labels selected with a reward adapted from SelfCite [7].

1 Introduction

Software often has to make a decision among fixed options: route a support ticket, classify the intent of a message, or check a claim against evidence. A chat model can do this, but its answer has to be generated and parsed, which adds latency and formatting failures. Jev [2] is a decision model of this kind, and models built in its style are called Jev-style decision models. They return a probability distribution over the options without generating free-form text, so a new task needs only a prompt and a list of options [27]. LLM-as-Jev [27] shows that a decoder language model already works as such a decision model, with no change to its architecture, tokenizer or vocabulary. The model sees the options numbered and answers with a number, which works for any number of options without training. Fine-tuning it to put its probability on the correct number, while keeping it close to the original model, helps most for small models and for tasks with many options. All the models it studies are decoders, which read the text from left to right.

A decision needs almost no generation: the model reads the input once and then scores the short identifiers of the options. Decoders are designed for the opposite case, long generation. Multi-query and grouped-query attention [1, 38] shrink the KV cache so that decoding can run with larger batches, and the parameters this removes are put back in the MLP layers. Du et al. [16] show what this costs when nothing is generated: the large intermediate tensors of the MLP layers cause periodic spikes in GPU memory during prefill, which limit the input length. An encoder has no generation step and keeps no KV cache, so this trade-off does not shape its design. We take these as reasons to try an encoder; at the scale of our models, Section 4.5 finds no such memory difference.

An encoder also reads the whole input in both directions. Its limitation was the head: a classifier head is fixed to one label set and must be trained again for each new task. A masked language model has no such limit. It already predicts a word at a masked position, so an option can be scored by asking whether the model fills a mask next to it with "yes" or "no". Recent encoders such as ModernBERT [42] and UltraBERT [32] are pretrained with masked

language modelling and can read long inputs, which decision tasks over documents need.

Reading in both directions matters for citing evidence as well. When a decoder serves many questions about one document, the document usually comes first, so that its KV cache can be reused across questions, as with prefix caching in vLLM [25] and RadixAttention in SGLang [50]. The cached document states are then not conditioned on the question. Our encoder reads the question and the document together, so the evidence head works on document states that are conditioned on the question.

We build an encoder decision model on this basis. It takes the same prompt and option list as LLM-as-Jev, puts one mask in front of each option, and scores all options in a single forward pass. It needs no new head and supports a different number of options in every row. Our contributions are:

- **An encoder decision model.** We fine-tune UltraBERT-380M [32] with one mask per option and the "yes" versus "no" prediction at each mask as the option's score. It is trained on the same data as LLM-as-Jev [27].
- **Training recipes for the encoder.** We compare four ways to train on the mask predictions, which differ in how an option is scored and how the scores are turned into probabilities over the options; one of them follows the expert gate of DeepSeek-V3 [14]. For each we measure accuracy, sensitivity to the order and removal of options, predicted label frequencies and calibration.
- **A comparison with decoders under matched data.** We fully fine-tune Qwen3-0.6B [34] on the same examples for the same single epoch, choose learning rates on a dev split, repeat the runs over seeds and evaluate on a test split excluded from model selection. We report accuracy, accuracy on external benchmarks, and time per decision, with two LoRA models [21] reproduced from LLM-as-Jev as references.
- **Predicting supporting evidence.** A decision model generates no text, so we train an auxiliary head on the same encoder to predict the annotated evidence for the chosen option; it is not evaluated as an explanation of how the model decides. Its labels come from human-marked evidence and from generated labels selected with a reward adapted from SelfCite [7]. We compare it with ContextCite [10] and AT2 [11] on agreement with human evidence on the test set, and on cost.

2 Method

2.1 Input format

Figure 1 shows the same row in both formats, built from the training data described in Section 3.1. The options are shuffled each time a row is used, so the position of the correct option carries no information.

Decoder. The prompt follows LLM-as-Jev. The user message holds the state, the question, the options numbered [1] to [K], and one instruction: select one option and answer only with its number. The assistant turn is filled in up to "Best answer: [", and the model has to continue with the number of its choice and a closing bracket. Qwen3's thinking mode is turned off. Each option k is therefore the token sequence of "k]", and the options form a trie over these sequences.

Encoder. The state and the question are the same. The instruction asks to mark the correct option with "yes" and every other option with "no", and the options are listed as "[k] `[M]` description", with the tokenizer's mask token in front of each description. There is no chat template and nothing is generated. One forward pass gives a vector of logits at every mask, and the score of option k comes from the logits of the tokens "yes" and "no" at mask k, each taken with a leading space, as it appears after another word.

2.2 Loss functions

Decoder: trie loss with KL anchors. At each node of the trie the model chooses among the tokens that are legal there. The tree loss is the negative log-probability of the gold option, which is the product of the choices along its

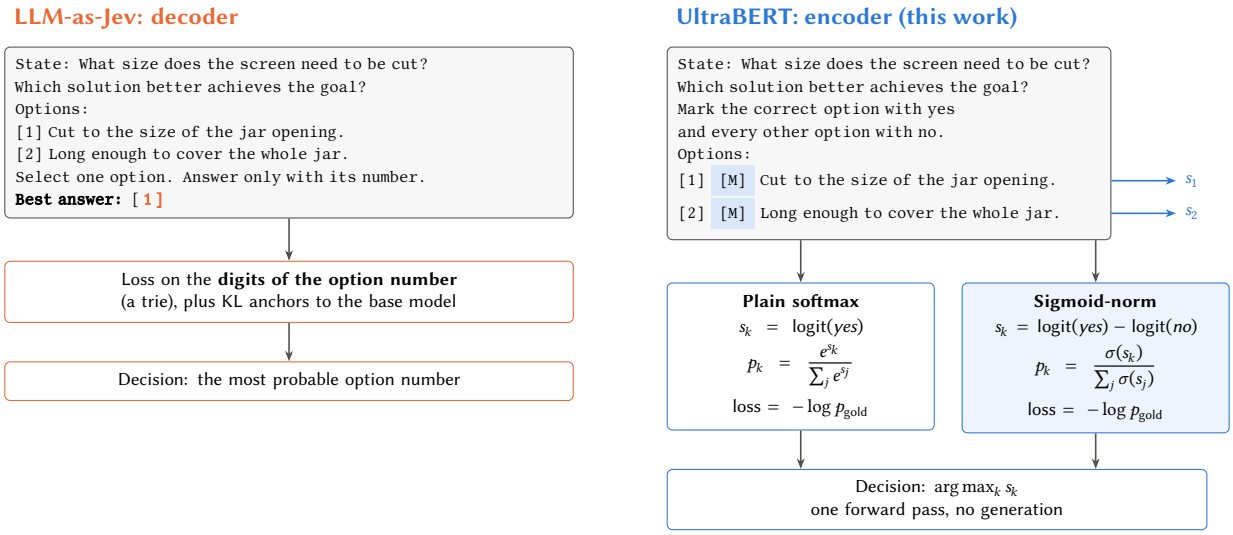


Figure 1. One row, two ways. The decoder reads the numbered options and writes the number of its answer; the loss is taken over the tokens of that number. The encoder gets a mask in front of each option and scores each option directly. Two of the four recipes are shown: the plain softmax over the options, which scores an option by the "yes" logit alone, and the sigmoid-normalised loss, which scores it by "yes" minus "no". Each option's score goes through a sigmoid, and the sum in the denominator keeps the options competing. The other two normalise "yes" minus "no" with a softmax and with a square-root softplus.

path, each normalised over the legal tokens only,

$$\mathscr{L}_{\text{tree}} = -\sum_t \log \frac{\exp z_t(g_t)}{\sum_{v \in \mathscr{V}_t} \exp z_t(v)}, \tag{1}$$

where z_t are the logits at step t, g_t the gold token and $\mathscr{V}_t$ the legal tokens. Three anchor terms keep the model close to its starting version p_0 [27]: $\mathscr{L}_{\text{mass}}$ keeps the total probability on the legal tokens, $\mathscr{L}_{\text{out}}$ keeps how the remaining probability is spread over the rest of the vocabulary, and $\mathscr{L}_{\text{pos}}$ keeps the next-token distribution at the start of the reply and after the closing bracket. They are KL divergences to p_0, computed over its 64 most likely tokens and one bucket for the rest. The loss is $\mathscr{L} = \mathscr{L}_{\text{tree}} + \lambda(\mathscr{L}_{\text{mass}} + \mathscr{L}_{\text{out}} + \mathscr{L}_{\text{pos}})$ with $\lambda = 1$.

Encoder recipes. Let $\ell_k(v)$ be the logit of token v at mask k, and let g be the gold option. A score s_k says how strongly the model accepts option k; the decision is the option with the highest score. Training raises the gold option's score relative to the other options, but no mask gets its own yes-or-no target. Thus, a chosen option can still prefer "no": for example, scores of -3 and -8 select the first option. We train the following four recipes:

- **Yes-logit softmax.** The score is the "yes" logit alone, and the options compete through one softmax:

$$s_k = \ell_k(yes), \qquad p_k = \frac{e^{s_k}}{\sum_{j=1}^{K} e^{s_j}}, \qquad \mathscr{L}_{\text{softmax}} = -\log p_g. \tag{2}$$

- **Yes-minus-no softmax.** The score measures the preference for "yes" over "no", followed by the same softmax:

$$s_k = \ell_k(yes) - \ell_k(no), \qquad p_k = \frac{e^{s_k}}{\sum_{j=1}^{K} e^{s_j}}, \qquad \mathscr{L}_{\text{softmax}} = -\log p_g. \tag{3}$$

- **Yes-minus-no sigmoid normalisation.** Each score first goes through a sigmoid, and the resulting values are normalised by their sum:

$$s_k = \ell_k(yes) - \ell_k(no), \qquad p_k = \frac{\sigma(s_k)}{\sum_{j=1}^{K} \sigma(s_j)}, \qquad \mathscr{L}_{\text{sig-norm}} = -\log p_g. \tag{4}$$

This has the form of the DeepSeek-V3 mixture-of-experts gate [14], but the options play the role of experts and are normalised over all K options, with no selection or balancing bias. Each $\sigma(s_k)$ lies between 0 and 1 and can be read as how strongly the model accepts option k on its own, although only the ratios enter p_k. The options therefore still compete, and the decision remains the option with the highest score because the sigmoid is increasing. The gradient with respect to s_k is

$$(1 - \sigma(s_k))\,(p_k - \mathbf{1}[k = g])\,. \tag{5}$$

Once an option's sigmoid is close to 1, its score hardly changes. We do not pair this normalisation with the "yes" logit alone: vocabulary logits are around 200 (Section 5.2), so every sigmoid is close to 1, every option receives probability $1/K$, and the gradient is negligible. The "yes"-minus-"no" score is much smaller and avoids this saturation. Nothing in the loss makes $\sigma(s_k)$ a probability that holds on its own; Section 4.4 checks what the trained model does.

- **Yes-minus-no square-root softplus.** To test whether the sigmoid's upper bound matters, we use $h(s) = \sqrt{\log(1 + e^s)}$:

$$s_k = \ell_k(yes) - \ell_k(no), \qquad p_k = \frac{h(s_k)}{\sum_{j=1}^{K} h(s_j)}, \qquad \mathscr{L}_{\text{sqrt-sp}} = -\log p_g. \tag{6}$$

For a clearly rejected option ($s \ll 0$), $h(s) \approx e^{s/2}$, so the options behave as in a softmax at twice the temperature. For a clearly accepted option ($s \gg 0$), $h(s) \approx \sqrt{s}$ keeps growing, whereas the sigmoid stops at 1. Its score therefore continues to receive a small gradient where the sigmoid would receive almost none.

3 Experimental setup

3.1 Training data

We follow LLM-as-Jev [27] and train on its Reason+Long mixture, with Nimble added. It has 13 datasets: intent routing (CLINC150 [26], MASSIVE [17], Bitext [6]), short reasoning (ReClor [45], LogiQA 2.0 [28], CosmosQA [22], PIQA [5], ANLI [29], ProofWriter [39]), long input (ContractNLI [23], MuSiQue [40], QuALITY [30]) and Nimble [4]. Each is turned into a state (the passages), a question and a list of options. We keep 1,000 training rows per dataset, 13,000 in total. Rows are filtered in this order:

1. **Duplicates.** Rows with the same input are collapsed to one, and rows with the same input but different answers are removed.
2. **Leakage between splits.** A training row is removed if its input, or the source document it comes from, also appears in the validation split.
3. **Benchmark overlap.** A training row is removed if it shares a 13-word n-gram, or an identical sentence of six or more words, with any evaluation benchmark.
4. **Length.** Prompts longer than 10,240 tokens, counted with the Qwen3.5-4B tokenizer, are dropped, never truncated.

The held-out rows are drawn from the validation split with the same duplicate and length filters, up to 500 per dataset and in whole source documents, giving 6,315 rows. We split them into a dev set of 1,300 rows, used to choose learning rates, and a test set of 5,015 rows that no choice depends on.

3.2 Models

Encoder. UltraBERT-380M [32] is fully fine-tuned with the format of Section 2.1 and each of the four recipes of Section 2.2.

Decoders. We fully fine-tune Qwen3-0.6B [34] with the loss of Section 2.2, with a frozen copy of the starting model as p_0. We also report two LoRA models [21], Qwen3-0.6B and Qwen3-4B, that we reproduced from the LLM-as-Jev paper on the same training set with the same loss. The encoder is fully fine-tuned, so the fully fine-tuned decoder is the matched comparison, and the LoRA models are references.

3.3 Implementation details

Every model is trained for one epoch with a batch of 32 rows. We sweep the learning rate for the decoder and for two encoder recipes, with one seed per value, and choose by accuracy on the dev set: $\{0.8, 1.5, 3, 5\} \times 10^{-5}$ for the encoder with the softmax over "yes", $\{0.8, 1.5, 2, 3\} \times 10^{-5}$ with the sigmoid-normalised loss, and $\{0.5, 1, 2\} \times 10^{-5}$ for the decoder. Both encoder sweeps choose 1.5×10^{-5}, which we also use for the other two recipes, without a sweep. The two best Qwen rates differ by 0.1 points on dev, and we use 1×10^{-5}. At the chosen rate we repeat the run with different seeds, which change the order of the rows and the option shuffling, and report the average.

The metric is accuracy. ALL is the mean accuracy over the 13 datasets, so that every dataset counts equally. On the test set we average each row's correctness over the seeds of a model and report ALL of this average. The encoder is scored with fp32 weights under bf16 autocast, because bf16 weights change its long-document results (Section 5.2). The Qwen checkpoints are scored with bf16 weights.

3.4 External benchmarks and cost

To see whether training on the mixture changes behaviour outside it, we score six benchmarks as decision rows: BoolQ [8], MMLU [20], MMLU-Pro [41], ARC-Challenge [9], Winogrande [36] and SciQ [43], with 500, 1200, 800, 600, 500 and 400 rows. The sample sizes follow LLM-as-Jev, whose samples are not released, so ours are seeded uniform draws. External is the mean accuracy over the six. These benchmarks are not used for any choice.

Time per decision is the latency of one request: we score the dev rows one at a time on one A100 and report the mean. This is the cost when a pipeline sends each item as it arrives; Appendix A gives the throughput when rows are batched. A call covers the forward pass and the scoring of the options, but not tokenisation; the first 20 rows warm up the GPU, which is synchronised around every call. For the decoder it includes the trie: Qwen3 writes each digit as a separate token, so the options numbered 10 and above need more than the first token after the input (Section 4.5). The dev rows average 1,100 to 1,200 tokens, depending on the tokenizer. The encoder is timed on the softmax checkpoint, which has the same architecture as the sigmoid-normalised one, in the scoring precision. The decoders are timed with the reproduced LoRA checkpoints in bf16, with the LoRA weights merged, which have the same architecture as the fully fine-tuned model.

4 Results

4.1 Encoder against decoder

Table 2 and Figure 2 give the main comparison. With the same training examples, one epoch and full fine-tuning, every encoder recipe is above Qwen3-0.6B (67.6% on the test set): 69.7% with the softmax over "yes", and 71.3%, 71.2% and 72.1% with the softmax, the square-root softplus and the sigmoid over "yes" minus "no". The encoder also has fewer parameters, 380M against 0.6B. On the six external benchmarks the encoder recipes reach 57.8% to 59.6%, against 56.1% for the decoder. These results concern UltraBERT-380M and Qwen3-0.6B under our training setup; the reproduced Qwen3-4B LoRA model, at 77.3%, remains more accurate overall.

4.2 Full fine-tuning against LoRA

The reproduced LoRA model of the same size reaches 63.0%, 4.6 points below the fully fine-tuned decoder. Under the two training configurations, full fine-tuning is therefore the stronger baseline, and we compare the encoder against it. The external accuracy of fully fine-tuned Qwen3-0.6B (56.1%) is not above that of its LoRA model (56.9%).

4.3 Recipes

Within the encoder recipes, changing the score from "yes" to "yes" minus "no" (both with the softmax) adds 1.6 points on the test set and raises external accuracy from 57.8% to 59.6%. Replacing the softmax with the sigmoid normalisation adds a further 0.8 test points (95% bootstrap interval over test rows [+0.2, +1.3]) and nothing on the external benchmarks. The square-root softplus, which has no upper bound, does not reproduce this gain (71.2% against 71.3% for the softmax). This comparison does not show why: the two functions also differ in their gradients and, for rejected options, the square-root softplus acts as a softmax at twice the temperature (Section 2.2). Section 5 compares the labels that each recipe predicts.

4.4 Sensitivity to option order and removal

We test whether models trained with different recipes differ in how much their answer depends on the rest of the option list. On the dev rows we score each row in four random option orders and once with one wrong option removed, and count how often the answer changes (Table 3). The normalisation alone cannot cause these changes: with the raw scores fixed, reordering or removing an option never changes the highest-scoring remaining option, under any of the normalisations. A change happens because the encoder reads the new option list and recomputes every score, so the test measures the trained model, not the normalisation function. Comparing the two softmax recipes shows the effect of the score; comparing the softmax and the sigmoid over "yes" minus "no" shows the effect of the normalisation used in training.

The score does most of the work: "yes" minus "no" raises external accuracy from 57.8% to 59.6%, lowers the answer changes when a wrong option is removed from 5.9% to 4.8%, and lowers the largest gradient norm from 842 to 139. When the options are reordered, the score and the sigmoid each lower the answer changes by about one point (15.6%, 14.6%, 13.7%). On top of the score, the sigmoid raises dev accuracy from 71.5% to 72.9%, lowers the calibration error from 0.027 to 0.023, and brings the frequency of each predicted label closer to its frequency among the gold answers, which the score alone moves further away; matching frequencies does not by itself mean the labels are assigned correctly (total variation 0.167, 0.213, 0.139). After training, the gold option's sigmoid averages only 0.049; far below 1, $\sigma(s) \approx e^s$, so the loss behaves almost like a softmax. The square-root softplus has the smallest largest gradient norm (104), one batch's value and not a full measure of stability, but it changes its answer most often when the options are reordered (16.6%).

4.5 Time and memory

A decoder needs the trie of option numbers to decide: from ten options on, the later digits tell the options apart. For a fair comparison we time the decoders with a scorer that reads the input once and then all trie nodes in one more forward pass on the shared KV cache, under a mask that lets each node see the input and its own earlier digits. It gives the same scores as the scorer used for evaluation (they agree to 10^{-4} in fp32), which copies the cache for every node and takes 92 ms and up to 8.6 GiB on Qwen3-0.6B.

One row at a time, the encoder takes 29 ms per decision, against 68 ms for Qwen3-0.6B and 124 ms for Qwen3-4B (Table 1). Reading the input alone takes the decoders 36 ms and 85 ms, and the rest is the second pass, over the trie. Memory is a different story: the trie adds almost nothing, and Qwen3-0.6B peaks at 2.2 GiB, against 2.8 GiB for the encoder with fp32 weights and 1.9 GiB with bf16 weights. At this scale and with inputs up to 10,240 tokens, the encoder is faster but does not use less memory. Appendix A gives the batched throughput.

5 Analysis

This section describes what we saw when training UltraBERT. Unless noted, the numbers are from the dev set with one seed per setting.

Table 1. Time and peak GPU memory per decision, scoring the dev rows one at a time on one A100. The decoders use bf16 weights and read the input, then the whole trie of option numbers in one more pass on the shared KV cache. Memory is what PyTorch allocates, in GiB, weights included; rows >4,096 tok.: mean peak over rows longer than 4,096 tokens; max: the largest peak over all rows.

Model	ms per decision	Weights	Peak, rows >4,096 tok.	Peak, max
UltraBERT-380M, fp32 weights	29	1.6	2.6	2.8
UltraBERT-380M, bf16 weights	26	0.8	1.5	1.9
Qwen3-0.6B	68	1.1	2.0	2.2
Qwen3-4B	124	7.6	8.9	9.3

Table 2. Matched comparison: average accuracy (%) over seeds. Test ALL is on the test set; Dev ALL and External are on the dev set and the six external benchmarks. Learning rates were chosen on dev. Bold: best below 1B parameters.

Model	Params	Dev ALL	Test ALL	External
UltraBERT-380M, softmax, yes	380M	71.2	69.7	57.8
UltraBERT-380M, softmax, yes − no	380M	71.7	71.3	**59.6**
UltraBERT-380M, sqrt-softplus, yes − no	380M	71.6	71.2	59.3
UltraBERT-380M, sigmoid-norm, yes − no	380M	**72.7**	**72.1**	59.5
Qwen3-0.6B, full FT	0.6B	69.0	67.6	56.1
Qwen3-0.6B, LoRA (reproduced)	0.6B	64.5	63.0	56.9
Qwen3-4B, LoRA (reproduced)	4B	79.3	77.3	73.5

5.1 What each part of the recipe changes

Table 4 changes one part of the recipe at a time, from the softmax over "yes" to the sigmoid-normalised loss. Scoring with the difference of the "yes" and "no" logits, with the softmax kept, raises the external accuracy by 2.0 points and the dev accuracy by 0.4 points; averaged over seeds, the score gives all of the external gain among the encoder recipes (57.8%, 59.6%, 59.5% in Table 3). Replacing the softmax with the sigmoid-normalised loss then raises dev accuracy by a further 1.3 points, and MuSiQue most of all, from 64% to 76%. Nimble falls from 55% with the plain score to 43% with the sigmoid-normalised loss in this one run, on 100 dev rows. On the test set, averaged over seeds, the sigmoid-normalised model is instead 3.7 points above the softmax on Nimble. We also tried the words "correct" and "incorrect" in place of "yes" and "no"; this did not help: 70.6% against 71.1% with the "yes" logit, and 70.1% against 71.5% for the difference with "incorrect".

The recipes also predict different labels. On MuSiQue the correct answer is "Yes" in 48% of the rows. The sigmoid-normalised models predict "Yes" for 47 to 60% of the rows, against 64% and 69% for the two softmax variants (dev rows). On ProofWriter, where "Unknown" is correct for a third of the rows, the softmax never predicts it and the sigmoid-normalised models predict it for 2 to 12%. This class stays largely unlearned for every variant.

5.2 Numerical precision

Two precision choices matter for the encoder. First, its masked-token logits are of the order of 200, where bf16 can only step by 1 or 2, so options would tie or swap; our implementation therefore computes the masked-token head, the logits, the "yes" minus "no" difference and the normalisation in fp32. Second, with that head in fp32, storing the weights in bf16 still changed MuSiQue accuracy by up to 8 points, presumably through rounding in the rest of the network; we did not trace where. We therefore evaluate the encoder with fp32 weights under bf16 autocast throughout. The decoders are scored with bf16 weights, and re-scoring a Qwen checkpoint in another precision moved its dev accuracy by at most 0.8 points.

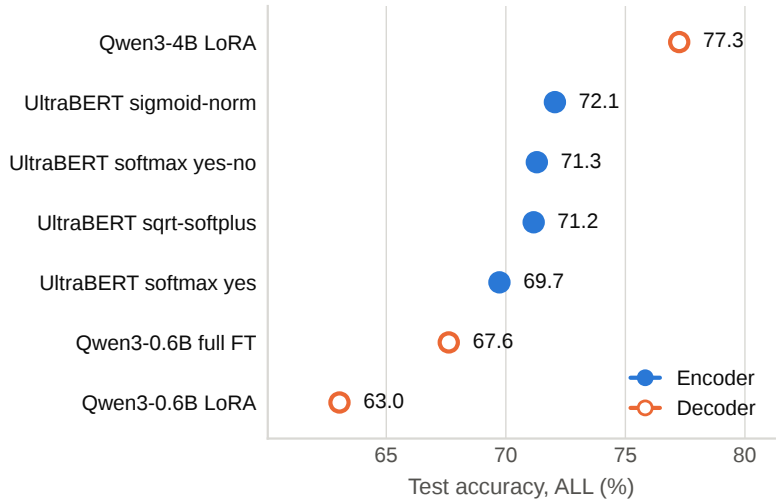


Figure 2. Test accuracy (ALL, the mean over 13 datasets, on the test set, never used for any choice), averaged over seeds; encoders filled, decoders hollow.

6 Predicting supporting evidence

A decision model writes no text, so it shows nothing a user can check. We add a small auxiliary head on the frozen encoder that predicts the document sentences annotated as supporting the chosen option, in the same forward pass. For each document token, it reads the token's hidden state and that of the chosen option's mask, and outputs the probability that the token is evidence. The head scores tokens; a sentence takes the highest score among its tokens. It is not evaluated as an explanation of how the encoder reaches its decision.

The head is trained on human-marked evidence where it exists, in ContractNLI, MuSiQue, Nimble and ProofWriter. For the other rows, a labelling model proposes sets of sentences for the gold answer, and we keep the set with the best reward adapted from SelfCite [7] (Figure 3, Appendix B).

Comparison. We compare with the encoder's average attention, AT2 [11] and ContextCite [10] on the 889 test rows with gold evidence. Nothing is tuned on these rows: the only setting, the threshold at which a word is marked, is tuned on the 267 dev rows with gold evidence. The head uses evidence supervision; the attribution baselines do not, and are fitted to the model's response to ablations instead. We report sentence average precision (AP) against the gold sentences, how often the top sentence is gold, and the time per row. For the baselines we also report LDS [10], the rank correlation between predicted and actual changes in the answer's probability when sentences are removed. We did not evaluate whether the heads are faithful to the model, so we report no LDS for them; their training target, annotated evidence, gives no reason to expect a high one. AT2 and ContextCite ran on the softmax encoder, so Table 5 has a head for each encoder; the like-for-like comparison is on the softmax encoder, and the head on the sigmoid-normalised encoder changes the encoder as well. The heads are conditioned on the option the encoder chose, and a wrong choice is scored against the evidence for the gold option, so the table measures the decision and the evidence together, not evidence localisation alone. The times per row are on these rows, not on the dev rows of Section 3.4, so they cannot be subtracted from the decision time to get the head's overhead.

Results. On the same encoder, the head has a higher sentence AP than ContextCite and AT2 (0.55 against 0.41 and 0.33) in 68 ms per row, against 1,860 ms for ContextCite. The lead comes from Nimble and ProofWriter; ContextCite is better than this head on ContractNLI and level on MuSiQue. The head on the sigmoid-normalised encoder reaches 0.68, and is level with or above ContextCite on those two datasets. The words it marks have a precision of 0.33 and a recall of 0.35, so they are a rough guide, not an exact span. Most of the head's accuracy does not come from the generated labels: on the softmax encoder, a head trained on human-marked evidence alone reaches 0.53, and adding the generated labels raises it to 0.55 (Appendix B).

Table 3. Sensitivity to option order and removal, on the dev set. Each row is scored in four random option orders; an answer change under reordering means the chosen option is not the same in all four. One wrong option, never the chosen one, is removed at random from rows with at least three options, and the answer is compared over the remaining options. Dev accuracy is on the first random order, so it differs slightly from Table 2, which keeps the original order. TV: total variation between the predicted and the gold frequencies, over the three datasets with a fixed label set (label) and over option positions (position). Calibration error: expected calibration error of the top option's probability. $|s_k|$: size of the raw option scores. Each metric is averaged over datasets, then over seeds. Bold: best in the row, higher for accuracy and lower for answer changes, TV and calibration error; the gradient norm and $|s_k|$ are not ranked.

	Softmax, "yes"	Softmax, "yes" − "no"	Sqrt-softplus, "yes" − "no"	Sigmoid-norm, "yes" − "no"
Dev accuracy (%)	71.4	71.5	71.1	**72.9**
External accuracy (%)	57.8	**59.6**	59.3	59.5
Answer changes, options reordered (%)	15.6	14.6	16.6	**13.7**
Answer changes, one wrong option removed (%)	5.9	**4.8**	5.4	4.9
Largest gradient norm in training	842	139	104	190
Label frequencies vs. gold (TV)	0.167	0.213	0.154	**0.139**
Position frequencies vs. gold (TV)	0.050	0.045	0.049	**0.041**
Calibration error, top option	0.038	0.027	0.032	**0.023**
Mean $\|s_k\|$	60.5	3.0	12.3	7.4

Table 4. Encoder variants on the dev rows, one seed (accuracy, %). The first row is the softmax with the "yes" logit, the setting used for the softmax results elsewhere. Bold: best in each column.

Variant	Dev ALL	External	MuSiQue	ContractNLI	Nimble
Softmax, "yes"	71.1	57.1	57.0	75.0	**55.0**
Softmax, "yes" − "no"	71.5	59.1	64.0	78.0	48.0
Sigmoid-norm, "yes" − "no"	**72.8**	**59.4**	**76.0**	**81.0**	43.0
Softmax, "correct"	70.6	56.8	48.0	77.0	52.0
Softmax, "correct" − "incorrect"	70.1	58.0	54.0	76.0	46.0

Limits. The head predicts annotated evidence, not what the model used; ContextCite has the highest LDS (0.64).

7 Related work

Jev-style decision models. Most of the community decision models compared in LLM-as-Jev [27] use a decoder backbone and read the logits of an option letter or label, with the backbone frozen or tuned with LoRA. LLM-as-Jev studies this readout and when fine-tuning helps. AnyJev reads a typed decision from one prefill and corrects the label prior and the effect of option order without training [46]. The Contrastive Language Model [24] embeds the state (the context and the question) and each candidate separately, with a frozen Qwen3-8B and two projection heads, and scores them by cosine similarity, so the candidate embeddings can be cached. Because the two are encoded apart, a candidate never attends to the context, and its training starts from tens of millions of pairs. All of these start from a decoder. We start from an encoder that reads the context, the question and all options together, and compare the two under the same training data.

Encoders that score options. Prompt-based classifiers use the masked-token head of an encoder with a verbalizer for a fixed set of labels [18, 37]. Two encoder decision models are listed on public Jev leaderboards [3]: Laya [12] and Von [31], both built on ModernBERT-large with one mask per option. Both add new layers on top of the encoder, and they are described in model cards. Laya trains its head with reinforcement learning on proper scoring rules, and Von uses a listwise softmax loss with a Brier term. We reuse the pretrained masked-token head and score each option by "yes" against "no", so there is no new head. We also compare four training recipes, match the decoders in

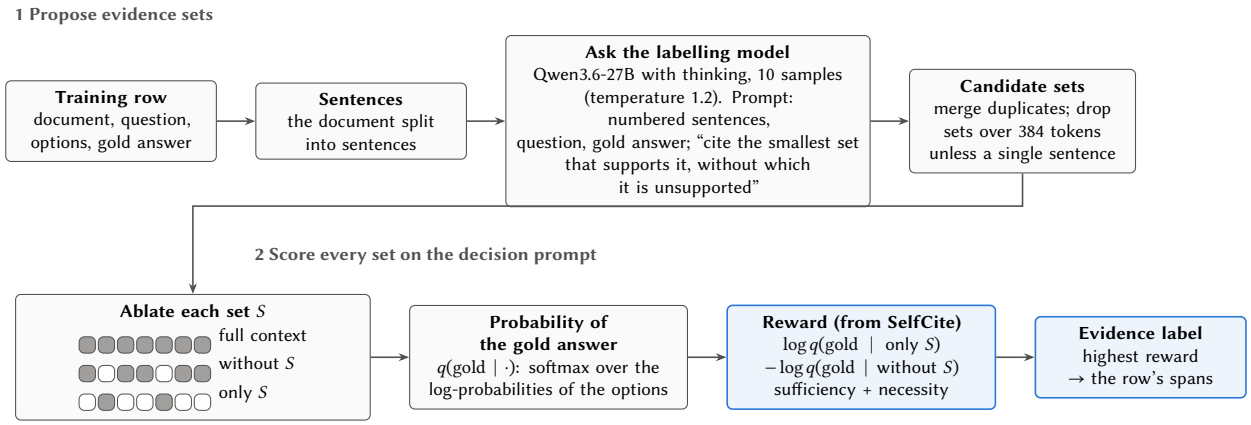


Figure 3. How evidence labels are generated for rows without human evidence. The labelling model, Qwen3.6-27B, proposes up to ten sets of sentences as evidence for the *gold* answer. The same model then answers the decision prompt three ways for each set S: with the full document, without S, and with only S: a good set is enough to keep the answer (sufficiency) and its removal loses the answer (necessity). The set with the highest reward becomes the label, which trains the evidence head. Unlike SelfCite, the answer scored is the gold answer and not the model's own response.

Table 5. Sentence evidence on the test rows with gold evidence. Sentence AP: average precision against the gold sentences, over all rows and per dataset (CNLI: ContractNLI, PW: ProofWriter). Top-1: share of rows where the top sentence is gold. LDS: how closely the scores follow the model's response to removed sentences; not evaluated for the heads. The first three methods and the first head use the encoder trained with the softmax. Each head is one training run. Bold: best in each column (lowest for time).

	Sentence AP							
Method	All	CNLI	MuSiQue	Nimble	PW	Top-1 (%)	LDS	ms per row
Average attention	0.25	0.07	0.20	0.34	0.38	14	0.09	69
AT2	0.33	0.30	0.28	0.30	0.42	23	0.15	71
ContextCite	0.41	0.52	0.33	0.33	0.44	44	**0.64**	1,860
Evidence head, softmax encoder	0.55	0.28	0.32	**0.96**	0.67	54	–	**68**
Evidence head, sigmoid-norm encoder	**0.68**	**0.55**	**0.52**	**0.96**	**0.68**	**74**	–	72

data and training, report a held-out test split, and add evidence.

Option letters, position and label bias. Reading the letter of the chosen option requires a model to bind the letter to the option text [35]. Models favour certain option labels, and their accuracy changes when the options are reordered [33, 49]. Corrections are usually applied after training, by calibrating with a content-free input [48] or by dividing out a label prior and averaging over rotations of the options [46]. We shuffle the options during training and measure how the recipes differ in the labels they predict.

Encoders and decoders. Controlled studies pretrain encoders and decoders with the same recipe, and find that encoders do well on classification and retrieval [44] and that masked language modelling suits tasks that need text representations [19]. A much larger decoder can still beat an encoder on classification [47]. Decoder designs such as multi-query and grouped-query attention target long generation [1, 38], and Du et al. [16] study what they cost when nothing is generated (Section 1). Modern encoders keep long inputs and fast inference [32, 42].

Attribution and evidence. ContextCite learns a surrogate model from many ablations of the context [10]. AT2 learns to weight attention heads, and needs a decoder pass [11]. SelfCite fine-tunes a model to cite, using the change in the answer probability when the cited text is removed or kept alone as the reward [7]. Rationale benchmarks such as ERASER compare selected evidence with human rationales [15]. We train an evidence head on the encoder with labels from human-marked evidence and from the SelfCite reward, which gives the evidence in one pass.

8 Conclusion

We built a decision model on an encoder, UltraBERT-380M, with one mask in front of each option, and compared four recipes for training it. With the same training examples, one epoch and full fine-tuning, every recipe is more accurate than Qwen3-0.6B trained with LLM-as-Jev (69.7% to 72.1% against 67.6% on the test set), and the encoder takes 29 ms per decision against 68 ms for the decoder, with similar peak memory. Among the encoder recipes, scoring an option by "yes" minus "no" instead of "yes" alone gives the larger test gain and all of the external gain. The sigmoid normalisation adds 0.8 test points and label frequencies closer to the gold ones; the square-root softplus does not, and our comparison does not show which property of the sigmoid is responsible. An auxiliary head on the frozen encoder predicts, in the same pass, the sentences annotated as support. On the softmax encoder and the test rows with gold evidence from four datasets, it ranks the annotated sentences higher on average than AT2 and ContextCite, which use no evidence labels, though ContextCite is better on ContractNLI.

The results have limits. They concern one encoder and one decoder at 0.4 to 0.6 billion parameters, and a 4B decoder is still more accurate overall, except on the intent datasets. We used one encoder and one training mixture, in English, and compared against one decoder family. The evidence head predicts annotated evidence, and we did not evaluate whether it reflects what the model used; ContextCite, which is fitted to the model's response to removed sentences, follows the model most closely. At 0.6B parameters and inputs up to 10,240 tokens, the decoder showed no prefill memory spike.

Two directions follow. The first is larger encoders, to see whether the advantage holds above one billion parameters. The second is to train the decision and the evidence together, so that the evidence head can also shape the encoder.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. arXiv:2305.13245.

[2] Diogo Almeida. Introducing system one models & Jev. TypeSafe AI blog, https://typesafe.ai/blog/introducing-system-one-models-and-jev, 2026.

[3] Benchmark Heaven. JevBench: Jev-class model benchmark (v1.4.2.2). https://benchmarkheaven.com/jev-models/, 2026.

[4] Bespoke Labs. Nimble. https://github.com/bespokelabsai/nimble, 2025.

[5] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of AAAI*, 2020.

[6] Bitext. Bitext customer support LLM chatbot training dataset. https://huggingface.co/datasets/bitext/Bitext-customer-support-llm-chatbot-training-dataset, 2023.

[7] Yung-Sung Chuang, Benjamin Cohen-Wang, Shannon Zejiang Shen, Zhaofeng Wu, Hu Xu, Xi Victoria Lin, James Glass, Shang-Wen Li, and Wen-tau Yih. SelfCite: Self-supervised alignment for context attribution in large language models. In *International Conference on Machine Learning*, 2025. arXiv:2502.09604.

[8] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of NAACL*, 2019.

[9] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[10] Benjamin Cohen-Wang, Harshay Shah, Kristian Georgiev, and Aleksander Madry. ContextCite: Attributing model generation to context. *arXiv preprint arXiv:2409.00729*, 2024.

[11] Benjamin Cohen-Wang, Yung-Sung Chuang, and Aleksander Madry. Learning to attribute with attention. *arXiv preprint arXiv:2504.13752*, 2025.

[12] Convai Innovations. Laya: an open non-autoregressive decision model. https://huggingface.co/convaiinnovations/laya, 2026.

[13] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

[14] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[15] Jay DeYoung, Sarthak Jain, Nazneen Fatema Rajani, Eric Lehman, Caiming Xiong, Richard Socher, and Byron C. Wallace. ERASER: A benchmark to evaluate rationalized NLP models. In *Proceedings of ACL*, 2020. arXiv:1911.03429.

[16] Kuntai Du, Bowen Wang, Chen Zhang, Yiming Cheng, Qing Lan, Hejian Sang, Yihua Cheng, Jiayi Yao, Xiaoxuan Liu, Yifan Qiao, Ion Stoica, and Junchen Jiang. PrefillOnly: An inference engine for prefill-only workloads in large language model applications. *arXiv preprint arXiv:2505.07203*, 2025.

[17] Jack FitzGerald et al. MASSIVE: A 1M-example multilingual natural language understanding dataset with 51 typologically-diverse languages. In *Proceedings of ACL*, 2023.

[18] Tianyu Gao, Adam Fisch, and Danqi Chen. Making pre-trained language models better few-shot learners. In *Proceedings of ACL*, 2021. arXiv:2012.15723.

[19] Hippolyte Gisserot-Boukhlef, Nicolas Boizard, Manuel Faysse, Duarte M. Alves, Emmanuel Malherbe, André F. T. Martins, Céline Hudelot, and Pierre Colombo. Should we still pretrain encoders with masked language modeling? *arXiv preprint arXiv:2507.00994*, 2025.

[20] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.

[21] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[22] Lifu Huang, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Cosmos QA: Machine reading comprehension with contextual commonsense reasoning. In *Proceedings of EMNLP-IJCNLP*, 2019.

[23] Yuta Koreeda and Christopher D. Manning. ContractNLI: A dataset for document-level natural language inference for contracts. In *Findings of EMNLP*, 2021.

[24] Jacky Kwok, Hangoo Kang, Tarun Suresh, Jon Saad-Falcon, Marco Pavone, Christopher Ré, and Azalia Mirhoseini. Contrastive language models: A system one model for fast and generalizable decision-making. Blog post, Stanford University and NVIDIA Research, 23 September 2026. https://contrastive-lm.notion.site, 2026.

[25] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[26] Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K. Kummerfeld, Kevin Leach, Michael A. Laurenzano, Lingjia Tang, and Jason Mars. An evaluation dataset for intent classification and out-of-scope prediction. In *Proceedings of EMNLP-IJCNLP*, 2019.

[27] Yinheng Li and Justin Wagle. LLM-as-Jev: LLMs are already Jev-style decision models—when and how to fine-tune them. *arXiv preprint arXiv:2610.02076*, 2026.

[28] Hanmeng Liu, Jian Liu, Leyang Cui, Zhiyang Teng, Nan Duan, Ming Zhou, and Yue Zhang. LogiQA 2.0—an improved dataset for logical reasoning in natural language understanding. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023.

[29] Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston, and Douwe Kiela. Adversarial NLI: A new benchmark for natural language understanding. In *Proceedings of ACL*, 2020.

[30] Richard Yuanzhe Pang, Alicia Parrish, Nitish Joshi, Nikita Nangia, Jason Phang, Angelica Chen, Vishakh Padmakumar, Johnny Ma, Jana Thompson, He He, and Samuel R. Bowman. QuALITY: Question answering with long input texts, yes! In *Proceedings of NAACL*, 2022.

[31] Victor Hugo Panisa. Von: a non-autoregressive decision model. https://huggingface.co/wfzyx/von, 2026.

[32] Jungwoo Park, Taewhoo Lee, and Jaewoo Kang. UltraBERT: Beyond the limit of BERT pre-training. *arXiv preprint arXiv:2610.15028*, 2026.

[33] Pouya Pezeshkpour and Estevam Hruschka. Large language models sensitivity to the order of options in multiple-choice questions. In *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024. arXiv:2308.11483.

[34] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[35] Joshua Robinson, Christopher Michael Rytting, and David Wingate. Leveraging large language models for multiple choice question answering. In *International Conference on Learning Representations*, 2023. arXiv:2210.12353.

[36] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale. In *Proceedings of AAAI*, 2020.

[37] Timo Schick and Hinrich Schütze. Exploiting cloze questions for few shot text classification and natural language inference. In *Proceedings of EACL*, 2021. arXiv:2001.07676.

[38] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[39] Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. ProofWriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of ACL-IJCNLP*, 2021.

[40] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 2022.

[41] Yubo Wang et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024.

[42] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, et al. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.

[43] Johannes Welbl, Nelson F. Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. In *Proceedings of the 3rd Workshop on Noisy User-generated Text*, 2017.

[44] Orion Weller, Kathryn Ricci, Marc Marone, Antoine Chaffin, Dawn Lawrie, and Benjamin Van Durme. Seq vs seq: An open suite of paired encoders and decoders. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2507.11412.

[45] Weihao Yu, Zihang Jiang, Yanfei Dong, and Jiashi Feng. ReClor: A reading comprehension dataset requiring logical reasoning. In *International Conference on Learning Representations (ICLR)*, 2020.

[46] Jiamu Zhang, Tianze Yang, Yucheng Shi, Evan Chen, Zixiang Nie, Kelly Wan, Liangjie Hong, Ninghao Liu, and Liang Wu. AnyJev technical report. *arXiv preprint arXiv:2610.00831*, 2026.

[47] Hang Zhao, Qile P. Chen, Yijing Barry Zhang, and Gang Yang. Advancing single and multi-task text classification through large language model fine-tuning. *arXiv preprint arXiv:2412.08587*, 2024.

[48] Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: Improving few-shot performance of language models. In *International Conference on Machine Learning*, 2021. arXiv:2102.09690.

[49] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. Large language models are not robust multiple choice selectors. In *International Conference on Learning Representations*, 2024. arXiv:2309.03882.

[50] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

A Implementation details

Table 6 lists the training settings. Both models train for one epoch, which is 406 steps at a batch of 32.

Table 6. Training settings. Learning rates are in Section 3.3.

	UltraBERT-380M	Qwen3-0.6B (full)
Optimiser	AdamW, $\beta = (0.9, 0.98)$, $\epsilon = 10^{-6}$	AdamW
Weight decay	0.1	0.01
Schedule	5% warmup, linear decay to 0	100 warmup steps, linear decay to 10%
Gradient clipping	1.0	1.0
Precision	fp32 weights, bf16 autocast	fp32 weights, bf16 autocast
Attention	PyTorch SDPA	PyTorch SDPA
Gradient checkpointing	yes	yes
Token limit	32,768	16,384

The encoder settings are those of the UltraBERT model card, and we change only the learning rate. No training row exceeds either token limit, because the data are cut at 10,240 tokens (Section 3.1).

Batched throughput. When the dev rows are scored in batches, the encoder processes 75.5 rows per second, against 16.4 for Qwen3-0.6B and 6.5 for Qwen3-4B. The encoder uses packed variable-length FlashAttention-2 [13] and the decoders use length-sorted padded batches with the cache-copying trie scorer, so part of this gap comes from the padding and the copies.

B Evidence head

We train one head on each fine-tuned encoder (sigmoid-normalised and softmax, both at a learning rate of $1.5 \cdot 10^{-5}$); the softmax one matches the encoder used by AT2 and ContextCite.

Model. For document token t and option k, the head reads $[h_t; h_k; h_t \odot h_k]$, the final hidden states of the token and of option k's mask. A two-layer perceptron (512 hidden units, GELU, dropout 0.1) maps it to one logit, whose sigmoid is the probability that the token is evidence for option k.

Labels. A token is positive if it overlaps an evidence span. We use gold evidence where it exists (Nimble, ContractNLI, MuSiQue, ProofWriter) and generated spans elsewhere (Figure 3): ANLI, QuALITY, LogiQA 2.0, CosmosQA and the remaining rows of ContractNLI, MuSiQue and ProofWriter. Qwen3.6-27B with thinking gets the numbered sentences, the question and the gold answer, and is asked for the smallest set of sentences that supports the answer and without which it is unsupported. We sample ten replies (temperature 1.2, top-p 0.9), merge

identical sets, and drop sets over 384 tokens unless they are one sentence. The same model scores each set S on the decision prompt with the full context, without S and with only S ($1 + 2m$ prompts for m sets); $q(\text{gold} \mid \cdot)$ is the softmax over the options' summed token log-probabilities. The label is the set with the highest reward $\log q(\text{gold} \mid \text{only } S) - \log q(\text{gold} \mid \text{without } S)$. This is sufficiency, $\log q(\text{gold} \mid \text{only } S) - \log q(\text{gold} \mid \text{full})$, plus necessity, $\log q(\text{gold} \mid \text{full}) - \log q(\text{gold} \mid \text{without } S)$; the full-context term cancels, and we keep it only to report the two parts. The best set is always kept, with no minimum reward, so a row whose candidates are all poor still gets a label. There is no size penalty: the token cap limits the length of a set, but nothing makes it the smallest sufficient one. Unlike SelfCite, we score the gold answer, so the reward is informative only where the labelling model believes it. We did not check the generated labels against human evidence directly; the comparison at the end of this appendix is an indirect check. There are 7,778 labelled training rows (7,390 for training, 388 for validation), with 5.2% positive tokens.

Training. Token-level binary cross-entropy, with positives weighted by the negative-to-positive ratio (18.1), on the frozen encoder. The head is conditioned on the gold option in training and on the chosen option at test time. AdamW (learning rate 10^{-3}, weight decay 0.01), 16 rows per step, three epochs; we keep the epoch with the best validation token average precision.

Evaluation. All methods are scored on the test rows with gold evidence and at least two sentences. A word is highlighted when its probability is at least a global threshold, tuned for word F1 on the 267 dev rows with gold evidence, none of which are test rows: 0.79 for the sigmoid-normalised encoder and 0.74 for the softmax encoder. On the sigmoid-normalised encoder, the head's token average precision is 0.46 (0.17 for a random ranking), and 0.63 of its ten highest tokens are gold. On the softmax encoder, adding the generated labels to the gold ones raises sentence average precision from 0.53 to 0.55 and token average precision from 0.38 to 0.39.

C Additional results

Table 7 gives the test accuracy for each dataset, averaged over seeds.

Table 7. Test accuracy (%) per dataset, averaged over seeds. UB is UltraBERT-380M: UB yes is the softmax over each option's "yes" logit; UB yes−no is the softmax over "yes" minus "no"; UB sqrt-sp and UB sig-norm normalise "yes" minus "no" with the square-root softplus and the sigmoid, each divided by the sum over options (Section 2.2). Bold: best below 1B parameters.

Dataset	Rows	UB yes	UB yes−no	UB sqrt-sp	UB sig-norm	Qwen full	Qwen LoRA	Qwen-4B LoRA
CLINC150	400	**91.8**	91.5	91.7	**91.8**	86.3	79.8	89.8
MASSIVE	400	83.7	84.5	83.8	**84.9**	76.9	68.8	83.5
Bitext	400	**99.3**	**99.3**	**99.3**	**99.3**	96.0	94.8	97.2
ReClor	400	58.9	60.7	**61.8**	59.9	57.7	51.2	72.2
LogiQA 2.0	400	52.9	54.8	55.4	54.8	**57.0**	56.0	73.5
CosmosQA	400	74.8	74.2	75.1	**75.2**	59.9	55.0	76.0
PIQA	400	74.8	**77.5**	75.9	75.1	70.2	68.8	83.8
ANLI	400	73.1	72.6	**73.9**	73.6	63.7	61.0	74.0
ProofWriter	400	60.7	61.5	61.4	61.6	**63.3**	63.0	70.5
ContractNLI	393	73.8	77.4	74.7	**78.6**	76.1	65.6	79.1
MuSiQue	400	58.9	63.0	62.2	**69.8**	62.9	54.8	71.5
QuALITY	398	56.9	**62.1**	60.8	61.3	51.5	50.0	68.8
Nimble	224	47.0	47.8	49.1	50.7	**57.3**	50.9	64.3

D Examples

The examples show the decision of the sigmoid-normalised encoder and the token scores of its evidence head for one row of each of three datasets. The rows are the first test row of each dataset in the evidence study that has at

most 1,400 tokens; MuSiQue has none. Only the tokens that the head scores at or above its dev-tuned threshold of 0.79 for the chosen option are highlighted, darker for a higher probability, and tokens inside the gold evidence are underlined. The encoder chooses the correct option in all three rows; the rows were picked by the rule above and not by outcome.

ContractNLI. 1,338 tokens. The model chooses option 1, which is **correct**. Question: Statement: "Receiving Party shall not reverse engineer any objects which embody Disclosing Party's Confidential Information." Does the contract entail the statement, contradict it, or not mention it?

k	Probability	Option	
1	85%	The contract entails the statement.	gold, chosen
2	0%	The contract contradicts the statement.	
3	15%	The contract does not mention it.	

Contract MUTUAL NONDISCLOSURE AGREEMENT Effective Date: 12/10/98 This Agreement governs the disclosure of information by and between Yahoo! Inc., a California corporation, and Restrac, Inc. ("Participant"). 1. The "Confidential Information" is that confidential, proprietary, and trade secret information being disclosed by the disclosing party described as (please be specific): (a) Yahoo Confidential Information (owned by Yahoo and any of its affiliates): Resume and Employment Related. (b) Participant Confidential Information: Resume and Employment Related. 2. Except as set forth in this Section 2, all Confidential Information shall be in tangible form and shall be marked as Confidential or proprietary information of the disclosing party. If the Confidential Information is disclosed orally or visually, it shall be identified as such at the time of disclosure and confirmed in a writing to the recipient within thirty (30) days of such disclosure. 3. Each of the parties agrees that it will not make use of, disseminate, or in any way disclose any Confidential Information of the other party to any person, firm or business, except to the extent necessary for negotiations, discussions and consultations with personnel or authorized representatives of the other party and any purpose the other party may hereafter authorize in writing. Each of the parties agrees that it shall disclose Confidential Information of the other party only to those of its employees who need to know such information and who have previously agreed, either as a condition to employment or in order to obtain the Confidential Information, to be bound by terms and conditions substantially similar to those of this Agreement. 4. There shall be no liability for disclosure or use of Confidential Information which is (a) in the public domain through no fault of the receiving party (b) rightfully received from a third party without any obligation of confidentiality, (c) rightfully known to the receiving party without any limitation on use or disclosure prior to its receipt from the disclosing party, (d) independently developed by the receiving party, (e) generally made available to third parties without any restriction on disclosure, or (f) communicated in response to a valid order by a court or other governmental body, as otherwise required by law, or as necessary to establish the rights of either party under this Agreement (provided that the party so disclosing has provided the other party with a reasonable opportunity to seek protective legal treatment for such Confidential Information). 5. "Residual Information" shall mean any Confidential Information of the disclosing party which may be retained in intangible form in the minds of those individuals of the receiving party who have had proper access to such Confidential Information. Notwithstanding anything else in this Agreement, the receiving party shall be free to use any Residual Information for any purpose whatsoever, including, without limitation, the development of its own products, provided that such party shall not be entitled to disclose Residual Information to any third parties unless such disclosure is in the course of, or as part of, any disclosure of its own products or their development. 6. Each of the parties agrees that it shall treat all Confidential Information of the other party with the same degree of care as it accords to its own Confidential Information and each of the parties represents that it exercises reasonable care to protect its own Confidential Information. 7. Each of the parties agrees that it will not modify, reverse engineer, decompile, create other works from, or disassemble any software programs contained in the Confidential Information of the other party unless otherwise specified in writing by the disclosing party. 8. All materials (including, without limitation, documents, drawings, models, apparatus, sketches, designs and lists) furnished to one party by the other, and which are designated in writing to be the property of such party, shall remain the property of such party and shall be returned to it promptly at its request, together with any copies thereof. 9. This Agreement shall govern all communications between the parties that are made during the period from the effective date of this Agreement to the date on which either party receives from the other written notice that subsequent communications shall not be so governed, provided, however that each party's obligations under Sections 2 and 3 with respect to Confidential Information of the other party which it has previously received shall continue unless and until such Confidential Information falls within Sections 4 or 5. 10. Neither party shall communicate any information to the other in violation of the proprietary rights of any third party. Neither party acquires any licenses under any intellectual property rights of the other party under this Agreement. This Agreement shall be governed in all respects by the laws of the United States of America and by the laws of the State of California as such laws are applied to agreements entered into and to be performed entirely within California between California residents. This Agreement may only be changed by mutual agreement of authorized representatives of the parties in writing. All notices or reports permitted or required under this Agreement shall be in writing and shall be by personal delivery, telegram, telex, telecopier, facsimile transmission or by certified or registered mail, return receipt requested, and shall be deemed given upon personal delivery, five (5) days after deposit in the mail, or upon acknowledgment of receipt of electronic transmission. Notices shall be sent to the addresses set forth at the end of this Agreement or such other address as either party may specify in writing. IN WITNESS WHEREOF, the parties have executed this Agreement in duplicate as of the date first written above. <TABLE> <S> <C> YAHOO! INC. ———————————— By: /s/ Matt Rowlen By: /s/ M. J. Fahey ———————————- ———————————— Printed Name: Matt Rowlen Printed Name: M. J. Fahey ———————— ———————— Title: Manager of Business Development Title: President ———————————- ———————————— Address: Yahoo! Address: Lexington, MA ———————————- ———————————- ———————————- ———————————- ———————————- ———————————- </TABLE>

Nimble. 558 tokens. The model chooses option 3, which is **correct**. Question: Apply this rubric: HIGH—route is correct, custody is intact, and the claimant supplies both a unique identifier and one undisclosed feature without a material conflict; release is permitted. MEDIUM—route is correct and the claim is plausible, but either required identifying element is missing or custody has a gap; hold for more evidence. LOW—route is wrong or a material item, date, location, or identifier conflicts; do not release. Weather, staff clothing, and similar incidental observations are distractors, not material conflicts.

k	Probability	Option	
1	6%	Do not release: LOW confidence because the route is incorrect or material facts conflict with the intake record.	
2	35%	Send to the Transit Depot Vault but hold the backpack: MEDIUM confidence because an identifying requirement is missing or custody is incomplete.	
3	60%	Send to the Transit Depot Vault and release to Mara: HIGH confidence because custody is intact and both required identifying elements match without material conflict.	gold, chosen

Context A navy canvas backpack was found on Route 18 bus 604 at 18:20 on 12 May. Transit operator Imani Cole sealed it as T-884 and delivered it to the civic hall receptionist, who transferred the unbroken seal to lost-property custodian Arun Vale without any gap in custody.

Facts The backpack sealed as T-884 was found in the transit vehicle Route 18 bus 604. The custody chain for backpack T-884 has no gap between Imani Cole sealing it and Arun Vale receiving it from the civic hall receptionist. The seal on backpack T-884 remained unbroken through Arun Vale's receipt of it. The serial patch recorded for backpack T-884 is a unique identifier for that backpack. The serial-patch value recorded for backpack T-884 in the intake log is SP-7742. Mara Venn supplied the serial-patch value SP-7742 when she filed her claim. The hidden-feature detail recorded for backpack T-884, a brass key inside a torn inner-lining pocket, was not publicly disclosed before Mara Venn supplied it. Mara Venn supplied a matching brass-key hidden-feature detail. All other material items, dates, times, locations, and identifiers Mara Venn supplied agree with the intake record.

Policy Policy routes property found in transit vehicles to the Transit Depot Vault, regardless of where it was handed in.

ProofWriter. 208 tokens. The model chooses option 1, which is **correct**. Question: Statement: "Gary is cold." Based only on the theory, is the statement true, false, or unknown?

k	Probability	Option	
1	83%	True: the theory implies the statement.	gold, chosen
2	0%	False: the theory implies that the statement is false.	
3	17%	Unknown: the theory implies neither the statement nor its negation.	

Theory Charlie is blue. Charlie is quiet. Dave is blue. Dave is cold. Dave is young. Erin is blue. Erin is quiet. Erin is white. Erin is young. Gary is cold. Gary is quiet. Gary is young. All quiet things are furry. If Charlie is cold then Charlie is young. White things are cold. All young things are nice. If Gary is nice then Gary is young. If something is young then it is nice. If something is furry and blue then it is white. If something is young and furry then it is nice.